COMPLETE STUDY NOTES

Introduction to Generative AI

8 Chapters · Every Topic Explained Simply

With Formulas, Examples, Tables & Diagrams

Easy English Language Throughout

ML Foundations | Data Preprocessing | Linear Models

Model Evaluation | Decision Trees | Neural Networks

NLP | Transformers & LLMs

Exam Preparation Edition · March 2026

Contents

1 Foundations of Machine Learning **3**
- 1.1 What is Artificial Intelligence? . . . 3
- 1.2 AI vs. Machine Learning vs. Deep Learning . . . 3
- 1.3 ML vs. Traditional Programming . . . 4
- 1.4 Types of Learning . . . 4
  - 1.4.1 Supervised Learning — Learning with a Teacher . . . 4
  - 1.4.2 Unsupervised Learning — Finding Hidden Patterns . . . 5
  - 1.4.3 Semi-Supervised Learning — Best of Both Worlds . . . 5
  - 1.4.4 Reinforcement Learning — Learning by Trial and Error . . . 5
- 1.5 Types of ML Tasks . . . 6

2 Data Preprocessing **7**
- 2.1 Handling Missing Values . . . 7
- 2.2 Feature Scaling . . . 7
  - 2.2.1 Min-Max Normalisation (Rescaling) . . . 8
  - 2.2.2 Standardisation (Z-score Scaling) . . . 8
- 2.3 Data Encoding Methods . . . 8
  - 2.3.1 Label Encoding . . . 8
  - 2.3.2 One-Hot Encoding . . . 9

3 Supervised Learning: Linear & Logistic Models **10**
- 3.1 Linear Regression . . . 10
  - 3.1.1 Model Formulation . . . 10
  - 3.1.2 The Assumptions of Linear Regression (LINE) . . . 10
  - 3.1.3 Limitations of Linear Regression . . . 11
- 3.2 Logistic Regression . . . 11
  - 3.2.1 The Sigmoid Function . . . 11
  - 3.2.2 Model Formulation . . . 11
  - 3.2.3 Limitations of Logistic Regression . . . 12

4 Machine Learning Model Evaluation **13**
- 4.1 Bias and Variance . . . 13
- 4.2 Classification Metrics . . . 13
  - 4.2.1 The Confusion Matrix . . . 14
- 4.3 Regression Metrics . . . 14
- 4.4 Cross-Validation . . . 15

5 Decision Trees **16**
- 5.1 What is a Decision Tree? . . . 16
- 5.2 How Does the Tree Choose Which Feature to Split On? . . . 16
  - 5.2.1 Gini Impurity . . . 16
  - 5.2.2 Information Gain (Entropy) . . . 17
- 5.3 Advantages & Disadvantages of Decision Trees . . . 17
- 5.4 Regression Trees . . . 17

6 Introduction to Neural Networks **19**

6.1 Architectures . . . 19
6.1.1 Single Cell Perceptron . . . 19
6.1.2 Multi-Layer Perceptron (MLP) . . . 19
6.2 Backpropagation — How Neural Networks Learn . . . 20
6.3 Activation Functions . . . 20
6.4 Recurrent Neural Networks (RNNs) . . . 21

7 Natural Language Processing (NLP) 23
7.1 Text Preprocessing & Tokenization . . . 23
7.2 Word Embeddings — Turning Words into Numbers . . . 24
7.2.1 Bag of Words (BoW) . . . 24
7.2.2 TF-IDF . . . 24
7.2.3 Word2Vec . . . 24
7.3 Sequential Modelling . . . 25
7.4 Language Model Foundations . . . 25

8 LLM Architecture & Responsible AI 26
8.1 Transformers — The Architecture Behind Modern AI . . . 26
8.1.1 Why Transformers? The Problem with RNNs . . . 26
8.1.2 Self-Attention Mechanism . . . 26
8.1.3 Multi-Head Attention . . . 27
8.1.4 Full Transformer Architecture . . . 27
8.2 Responsible AI & Fairness Principles . . . 27
8.3 Bias in AI — Where Does it Come From? . . . 28
8.3.1 Bias Mitigation Strategies . . . 28
8.4 Misinformation Prevention . . . 29

Chapter 1

Foundations of Machine Learning

1 Foundations of Machine Learning

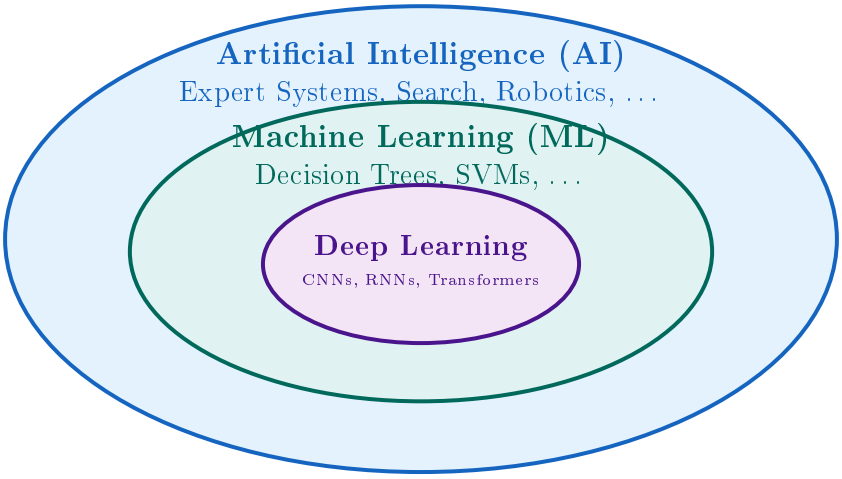
1.1 What is Artificial Intelligence?

Imagine teaching a child to recognise cats. You show the child many pictures of cats, and after a while the child can say "that's a cat!" even when seeing a new cat photo. **Artificial Intelligence (AI)** is about building computer systems that can do the same kind of thing — learn from experience and make smart decisions.

AI is the broad field of computer science that focuses on building machines capable of performing tasks that normally require human intelligence, such as understanding language, recognising images, and making decisions.

1.2 AI vs. Machine Learning vs. Deep Learning

Think of these three as **nested circles** — like Russian dolls:



Aspect	AI	Machine Learning	Deep Learning
Scope	Widest field	Subset of AI	Subset of ML
Data Needed	Low to High	Medium	Very High
Feature Engineering	Done by hand	Semi-manual	Automatic
Interpretability	Usually high	Medium	Low (black box)
Real Examples	Chess engines	Spam filters	ChatGPT, GPT-4

1.3 ML vs. Traditional Programming

Traditional Programming: You write rules → Computer follows them.
Machine Learning: You give examples → Computer *learns* the rules itself.

Example: Suppose you want to detect spam email.

- **Traditional:** You write "if email contains the word 'free money', mark as spam." — This is rigid and breaks easily.
- **ML:** You show the computer 10,000 spam emails and 10,000 normal emails. It learns the patterns on its own — much more powerful!

Traditional: **Data + Rules ⟶ Output**
Machine Learning: **Data + Output ⟶ Rules (= Model)**

1.4 Types of Learning

1.4.1 Supervised Learning — Learning with a Teacher

The training data has **labels** (correct answers). The model learns to map inputs to those correct answers. It is like a student who studies past exam papers with the answer key.

- **Classification:** Input = email text, Label = Spam / Not Spam
- **Regression:** Input = house size, Location, Label = Price ($250,000)

1.4.2 Unsupervised Learning — Finding Hidden Patterns

The training data has **no labels**. The model must discover structure on its own — like sorting a pile of coloured balls without being told which colours exist.

- **Clustering:** Group customers into "budget shoppers", "luxury buyers", etc. — no one told the model these groups existed.
- **Dimensionality Reduction (PCA):** Compress 100 features down to 2 for visualisation.

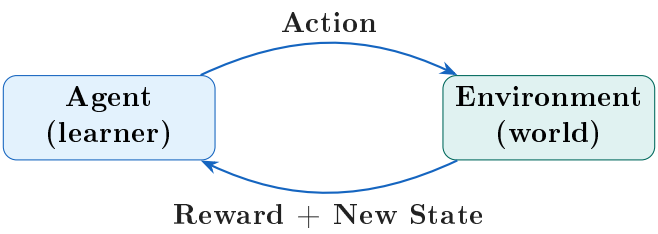
1.4.3 Semi-Supervised Learning — Best of Both Worlds

You have a **small amount of labelled data** and a **large amount of unlabelled data**. The model uses both. This is practical because labelling data is expensive and time-consuming.

Medical imaging: A hospital has 200 labelled X-rays (an expensive doctor reviewed them) and 50,000 unlabelled X-rays. Semi-supervised learning uses all of them.

1.4.4 Reinforcement Learning — Learning by Trial and Error

An **agent** takes **actions** in an **environment**, receives **rewards** (good) or **penalties** (bad), and learns to maximise its total reward over time.



- **AlphaGo:** Learned to play the board game Go by playing millions of games against itself.
- **Self-driving car:** Receives rewards for safe driving, penalties for crashes.

1.5 Types of ML Tasks

Task	Output Type	Loss Function	Example
Classification	Discrete label (class)	Cross-Entropy	Spam / Not Spam
Regression	Continuous number	MSE / MAE	Predicting house price
Clustering	Group membership	Inertia, Silhouette	Customer segments

Chapter Summary

Chapter 1 in one sentence: AI is the big field; ML is a subset that learns from data; DL is a further subset using neural networks. Learning can be supervised (with labels), unsupervised (no labels), semi-supervised (few labels), or reinforcement (learn by reward).

Chapter 2
Data Preprocessing

2 Data Preprocessing

Before feeding data to any model, we must **clean and prepare** it. Think of data like vegetables before cooking — you wash, cut, and season them first.

2.1 Handling Missing Values

Real-world data almost always has **gaps** (missing values). We cannot simply ignore them — they will cause errors. There are several strategies:

Strategy	What it does	When to use
Deletion	Remove rows/columns with missing values	Only when $< 5\%$ values are missing
Mean Imputation	Fill NaN with the column average	Numerical data, no heavy outliers
Median Imputation	Fill NaN with the column median	Numerical data with outliers
Mode Imputation	Fill NaN with most frequent value	Categorical data
KNN Imputation	Use similar rows to estimate value	Small datasets with structure

Median vs Mean: If your column has outliers (e.g., a few people earning millions in a salary column), use the **median** — it is not affected by extreme values. The mean gets "pulled" toward outliers.

2.2 Feature Scaling

Imagine a dataset with two features: **Age** (20–80) and **Income** (20,000–500,000). Income has much larger numbers. Many algorithms (like KNN, Linear Regression, Neural Networks) will be dominated by Income and almost ignore Age. Scaling fixes this by putting all features on the same playing field.

2.2.1 Min-Max Normalisation (Rescaling)

This method squeezes all values into the range **[0, 1]**.

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Example: If ages are 20–80, then age 50 becomes $\frac{50-20}{80-20} = \frac{30}{60} = 0.5$

2.2.2 Standardisation (Z-score Scaling)

This method transforms data to have **mean = 0** and **standard deviation = 1**.

$$x' = \frac{x - \mu}{\sigma}$$

where μ = column mean, σ = column standard deviation.
A value of $x' = 1.5$ means "1.5 standard deviations above the mean".

	Min-Max Normalisation	Standardisation
Output range	[0, 1]	$(-\infty, +\infty)$, mean 0
When to use	Neural networks, KNN	SVM, PCA, Linear Regression
Sensitive to outliers?	YES — outliers compress other values	Less sensitive

Always **fit** the scaler on **training data only**, then **apply** (transform) it to test data. Never fit on test data — that causes **data leakage** (the model "peeks" at test information before evaluation).

2.3 Data Encoding Methods

Machine learning models work with **numbers**, but many features in real datasets are **text** (e.g., "Red", "Blue", "Green" or "London", "Paris", "Delhi"). We must convert these to numbers.

2.3.1 Label Encoding

Assigns an integer to each category: `Red = 0, Green = 1, Blue = 2`.

Label encoding implies an **order** ($2 > 1 > 0$). For unordered (nominal) categories like colours or city names, this is **wrong** — the model might think "Blue > Red" is meaningful. Use only for **ordinal** data (e.g., Low=0, Medium=1, High=2).

2.3.2 One-Hot Encoding

Creates a **new binary column for each category**. No false ordering is introduced.

$$\text{Colour: Red} \rightarrow \begin{bmatrix}1 & 0 & 0\end{bmatrix}, \quad \text{Green} \rightarrow \begin{bmatrix}0 & 1 & 0\end{bmatrix}, \quad \text{Blue} \rightarrow \begin{bmatrix}0 & 0 & 1\end{bmatrix}$$

When using one-hot encoding, **drop one column** (use `drop='first'`) to avoid the **dummy variable trap** — having one column be a perfect combination of the others confuses the model.

Chapter Summary

Chapter 2 summary: Before modelling, always handle missing values (delete or impute), scale numeric features (standardise or normalise), and encode categorical features (label encoding for ordinal, one-hot for nominal).

Chapter 3

Supervised Learning: Linear & Logistic Models

3 Supervised Learning: Linear & Logistic Models

3.1 Linear Regression

Linear Regression answers: "Given some input numbers, predict a single output number." It finds the **best straight line** (or flat surface in higher dimensions) through the data.

3.1.1 Model Formulation

$$\hat{y} = \underbrace{\beta_0}_{\text{intercept}} + \underbrace{\beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n}_{\text{weighted features}}$$

In matrix notation: $\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\beta}$
The parameters $\boldsymbol{\beta}$ are found using **Ordinary Least Squares (OLS)**:

$$\boldsymbol{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

This minimises the **Residual Sum of Squares (RSS)**: $\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

Predicting house price:
$\hat{\text{Price}} = 50{,}000 + 300 \times \text{SquareFeet} + 20{,}000 \times \text{Bedrooms}$
A 1000 sq ft, 2-bedroom house: $\hat{\text{Price}} = 50{,}000 + 300{,}000 + 40{,}000 = \$390{,}000$

3.1.2 The Assumptions of Linear Regression (LINE)

For the model to be reliable, these four assumptions must hold:

- **L** **Linearity** — The relationship between inputs and output is actually linear. (Check: scatter plots should show a line pattern, not a curve.)
- **I** **Independence** — Each data point is independent of the others. (Violated in time-series data where yesterday's value affects today's.)
- **N** **Normality** — The prediction errors (residuals) should follow a bell curve.
- **E** **Equal Variance (Homoscedasticity)** — The size of errors should stay constant across all prediction values. (Violated if errors are small for small values but huge for large values.)

3.1.3 Limitations of Linear Regression

- Cannot capture **non-linear** relationships (e.g., price of gold, which goes up and down).
- Very sensitive to **outliers** (one extreme value can drag the entire line).
- Struggles when input features are strongly correlated (**multicollinearity**).
- Completely unsuitable for **classification** tasks.

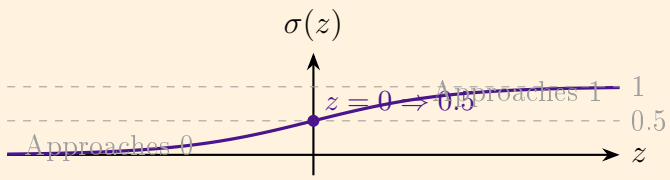
3.2 Logistic Regression

Despite its name, Logistic Regression is used for **classification** (not regression). It predicts the **probability** that something belongs to a class. "What is the chance this email is spam?" — 87% ⇒ spam.

3.2.1 The Sigmoid Function

The core idea: take the output of a linear equation and pass it through the **sigmoid** function to squeeze it into a probability between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}} \qquad \in (0, 1)$$



3.2.2 Model Formulation

Step 1 — Linear combination:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Step 2 — Sigmoid to get probability:

$$P(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Step 3 — Decision:

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1 \mid \mathbf{x}) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Loss function (Binary Cross-Entropy):

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

3.2.3 Limitations of Logistic Regression

- Assumes a **linear decision boundary** — fails when classes are not linearly separable.
- Sensitive to **outliers** in feature space.
- Requires **independent features** (no multicollinearity).
- Needs relatively **large samples** to be reliable.

Chapter Summary

Chapter 3 summary: Linear Regression predicts continuous numbers. Logistic Regression classifies (despite the name!) using the sigmoid function. Both assume linearity. Key difference: Linear uses MSE loss; Logistic uses cross-entropy loss.

Chapter 4

Machine Learning Model Evaluation

4 Machine Learning Model Evaluation

4.1 Bias and Variance

A model must be neither **too simple** (misses real patterns — *underfitting*) nor **too complex** (memorises noise — *overfitting*). We want it "just right."

Underfitting	**Just Right**	**Overfitting**
High Bias	Low Bias + Low Variance	High Variance

Model Complexity $\longrightarrow$

$$\text{Total Error} = \underbrace{\text{Bias}^2}_{\text{underfitting error}} + \underbrace{\text{Variance}}_{\text{overfitting error}} + \underbrace{\varepsilon}_{\text{irreducible noise}}$$

- **Bias** = how far your average predictions are from the truth (systematic error).
- **Variance** = how much predictions vary across different training sets (sensitivity).
- **Irreducible noise** = randomness in data we can never remove.

	High Bias (Underfitting)	**High Variance (Overfitting)**
Training error	High	Very low
Test error	High	High
Root cause	Model too simple	Model too complex
Fix	More features, complex model	Regularisation, more data, pruning
Example	Linear model on curved data	Deep tree memorising training set

4.2 Classification Metrics

4.2.1 The Confusion Matrix

A **confusion matrix** summarises how well your classifier performed by comparing predictions to actual labels:

	Predicted: Positive	Predicted: Negati
Actual: Positive	**TP** True Positive	**FN** False Negative (Type II Error)
Actual: Negative	**FP** False Positive (Type I Error)	**TN** True Negative

$$\textbf{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad \text{(correct predictions out of all)}$$

$$\textbf{Precision} = \frac{TP}{TP + FP} \quad \text{(of those I said were positive, how many really were?)}$$

$$\textbf{Recall} = \frac{TP}{TP + FN} \quad \text{(of all real positives, how many did I find?)}$$

$$\textbf{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad \text{(harmonic mean of P and R)}$$

- **Precision is critical** when a False Positive is costly: spam filter (legitimate email marked as spam is annoying).
- **Recall is critical** when a False Negative is costly: cancer detection (missing a real tumour is dangerous).
- **F1 Score** balances both — use when neither FP nor FN is clearly worse.
- **Accuracy** is misleading with imbalanced classes (e.g., 99% of data is class 0 — a model predicting always 0 gets 99% accuracy but is useless!).

4.3 Regression Metrics

$$\textbf{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i| \quad \text{(mean absolute error — robust to outliers)}$$

$$\textbf{MSE} = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \quad \text{(mean squared error — punishes big errors more)}$$

$$\textbf{RMSE} = \sqrt{MSE} \quad \text{(same units as the target variable — easier to interpret)}$$

$$\mathbf{R}^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} \quad \in (-\infty, 1] \quad (1 = \text{perfect}, 0 = \text{predicting the mean})$$

$$\textbf{Adjusted } \mathbf{R}^2 = 1 - \frac{(1 - R^2)(n-1)}{n - p - 1} \quad \text{(penalises adding useless features)}$$

$\mathbf{R^2}$ can be negative! If your model is worse than simply predicting the mean of y every time, $R^2 < 0$. Adjusted R^2 decreases when you add features that don't actually help.

4.4 Cross-Validation

A single 80/20 train/test split can be **lucky or unlucky** depending on which data points end up in test. Cross-validation gives a more **reliable, averaged estimate** of performance.

Fold 1	**Test**	Train	Train	Train	Train
Fold 2	Train	**Test**	Train	Train	Train
Fold 3	Train	Train	**Test**	Train	Train
Fold 4	Train	Train	Train	**Test**	Train
Fold 5	Train	Train	Train	Train	**Test**

Report average score across all 5 folds

1. Split data into k equal folds (usually $k = 5$ or $k = 10$).
2. For each fold i: train on the other $k - 1$ folds, test on fold i.
3. Final score $= \frac{1}{k}\sum_{i=1}^{k} \text{score}_i$

Chapter Summary

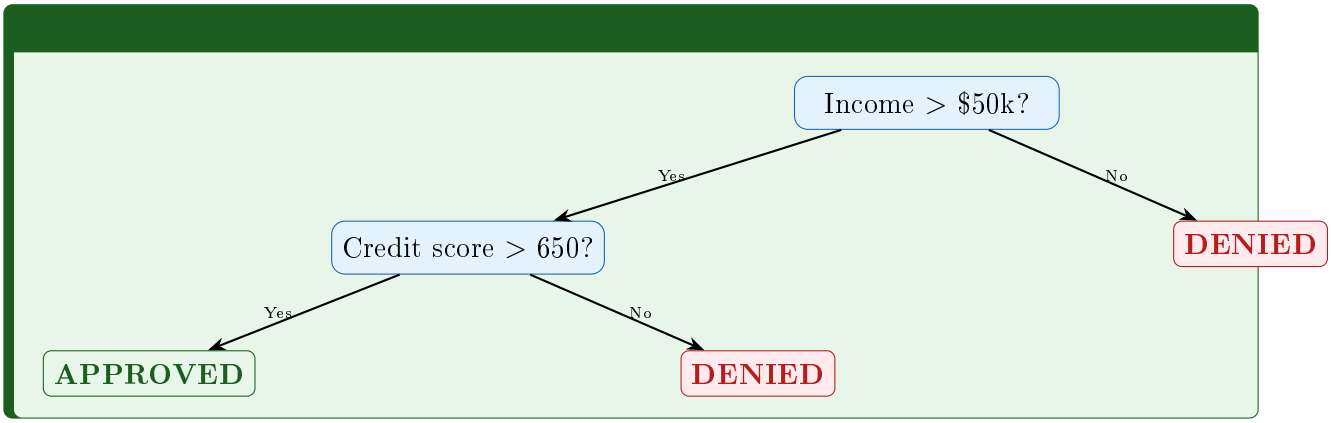
Chapter 4 summary: Evaluate classifiers with Precision, Recall, F1. Evaluate regressors with MAE, RMSE, R^2. Always check for bias/variance tradeoff. Use cross-validation for reliable estimates.

Chapter 5
Decision Trees

5 Decision Trees

5.1 What is a Decision Tree?

A **decision tree** is like the game of "20 Questions." You keep asking yes/no questions until you arrive at an answer.



- **Root Node:** The very first question (top of the tree).
- **Internal Node:** Any question node that has children.
- **Branch:** A path taken after answering a question.
- **Leaf Node:** A final answer node (no more splits). Contains a class label (classification) or a predicted value (regression).
- **Depth:** How many questions are asked from root to leaf.

5.2 How Does the Tree Choose Which Feature to Split On?

The tree chooses the feature (and split value) that **best separates** the classes — i.e., makes the resulting groups as **pure** as possible. We measure purity using **impurity measures**.

5.2.1 Gini Impurity

Gini impurity measures "how messy" a node is. If all samples in a node belong to the **same class**, Gini = 0 (perfectly pure). If samples are equally split between classes, Gini is at its maximum.

$$\text{Gini}(D) = 1 - \sum_{i=1}^{C} p_i^2$$

where p_i = fraction of samples belonging to class i in node D, and C = number of classes.

Example: A node with 3 cats and 7 dogs (10 total):
$p_{\text{cat}} = 0.3,\ p_{\text{dog}} = 0.7$
$\text{Gini} = 1 - (0.3^2 + 0.7^2) = 1 - (0.09 + 0.49) = 1 - 0.58 = \mathbf{0.42}$

A node with **all dogs**: $\text{Gini} = 1 - (0^2 + 1^2) = 0$ (perfectly pure!)

5.2.2 Information Gain (Entropy)

$$\text{Entropy}(D) = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

$$\text{Information Gain} = \text{Entropy(parent)} - \sum_{v} \frac{|D_v|}{|D|} \cdot \text{Entropy}(D_v)$$

We choose the feature with the **highest Information Gain** — it reduces uncertainty the most.

5.3 Advantages & Disadvantages of Decision Trees

Advantages

- Very **easy to understand** and visualise (white-box model)
- Needs **no feature scaling** (works with raw data)
- Handles both **numerical and categorical** features
- Can capture **non-linear** patterns
- Fast to predict (just follow branches)

- Prone to **overfitting** — deep trees memorise training data
- Sensitive to small changes in data (**high variance**)
- Greedy splitting — may miss the globally best tree
- Biased toward features with **many unique values**

The solution to decision tree overfitting: use a **Random Forest** — build hundreds of trees on random subsets of data and average their predictions. This dramatically reduces variance!

5.4 Regression Trees

Decision trees can also predict **continuous values** (not just classes).

Instead of minimising Gini impurity, they minimise **Mean Squared Error (MSE)** at each split. The predicted value at each leaf = the **mean of all training samples** that fall into that leaf.

Chapter Summary

Chapter 5 summary: Decision trees split data recursively using Gini or entropy. They are interpretable but overfit easily. Regression trees work the same way but minimise MSE. Random Forests fix the overfitting problem by averaging many trees.

Chapter 6

Introduction to Neural Networks

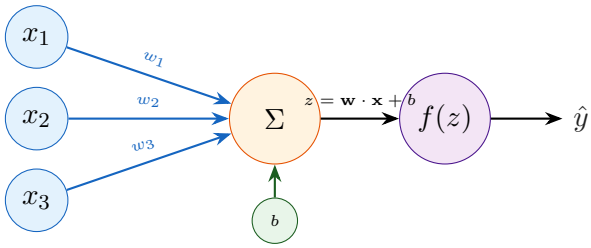
6 Introduction to Neural Networks

Neural networks are loosely inspired by the human brain, which has billions of neurons connected by synapses. A digital neuron takes several inputs, combines them, and fires an output signal.

6.1 Architectures

6.1.1 Single Cell Perceptron

The simplest possible neuron. It takes inputs, multiplies each by a weight (importance), adds them up, adds a bias, and passes the result through an **activation function**.

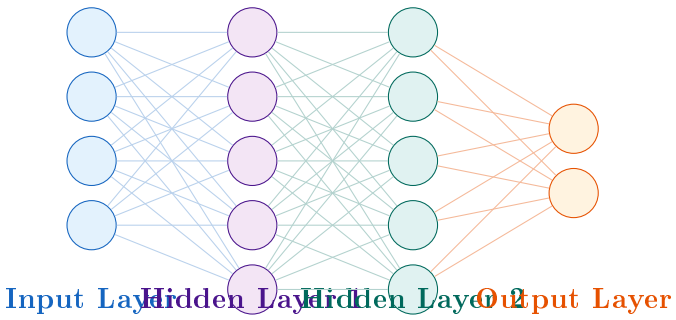


$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = \mathbf{w} \cdot \mathbf{x} + b$$

$$\hat{y} = f(z) \quad \text{where } f \text{ is the activation function}$$

6.1.2 Multi-Layer Perceptron (MLP)

A **deep** network with one or more **hidden layers** between inputs and outputs. This is what most people mean by "a neural network."



An MLP with **at least one hidden layer** and sufficient neurons can approximate **any continuous function** to any desired accuracy. This is why neural networks are so powerful.

6.2 Backpropagation — How Neural Networks Learn

Backpropagation is the algorithm that tells each weight in the network: "You made the error bigger — decrease your value" or "You reduced the error — increase your value." It does this using **calculus (chain rule)**.

Step 1 — Forward Pass: Feed input $\mathbf{x}$ through the network, compute output $\hat{y}$, compute loss $\mathcal{L}$.

Step 2 — Backward Pass (Backprop): Use chain rule to compute $\frac{\partial \mathcal{L}}{\partial w}$ for every weight w.

Step 3 — Weight Update (Gradient Descent):

$$w \leftarrow w - \eta \cdot \frac{\partial \mathcal{L}}{\partial w}$$

where η (eta) is the **learning rate** — how big each update step is.

If the learning rate η is **too large**, training diverges (bounces around). If **too small**, training is painfully slow. Common starting values: **0.001** with Adam optimiser.

6.3 Activation Functions

Activation functions introduce **non-linearity** into the network. Without them, stacking many layers would still just be a linear function (pointless!).

Function	Formula	Range	When to Use
Sigmoid	$\frac{1}{1+e^{-x}}$	$(0, 1)$	Binary classification output layer
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	$(-1, 1)$	Hidden layers (zero-centred, better than sigmoid)
ReLU	$\max(0, x)$	$[0, \infty)$	**Default choice** for hidden layers in deep nets
Leaky ReLU	$\max(0.01x,\ x)$	$(-\infty, \infty)$	Fixes "dying ReLU" problem
Softmax	$\frac{e^{x_i}}{\sum_j e^{x_j}}$	$(0, 1)$, sums to 1	Multi-class output layer

When using Sigmoid or Tanh in very deep networks, gradients become extremely tiny as they travel backward through layers. Weights at early layers **barely update** — training stalls. **Solution: Use ReLU** in hidden layers, which does not saturate for positive inputs.

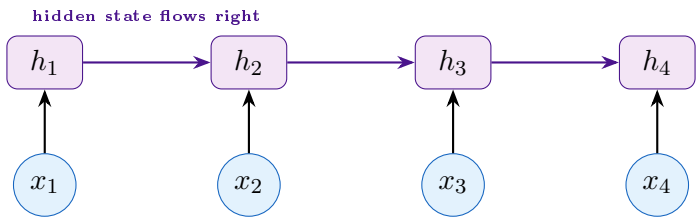
6.4 Recurrent Neural Networks (RNNs)

Standard MLP processes inputs independently — it has no memory. But language, time-series data, and audio are **sequential** — what came before matters.

An RNN processes sequences step by step, maintaining a **hidden state** h_t that carries information from previous steps:

$$h_t = f(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b)$$

Think of h_t as the network's "short-term memory" at time step t.



Basic RNNs suffer from the **vanishing gradient problem over long sequences** — they forget things from far back. **LSTM (Long Short-Term Memory)** and **GRU (Gated Recurrent Unit)** solve this with special **gating mechanisms** that control

what to remember and what to forget.

Chapter Summary

Chapter 6 summary: Neural networks are composed of layers of neurons. Deep networks (MLP) can learn complex patterns. Backpropagation + gradient descent trains them. ReLU is the go-to activation. RNNs handle sequences; LSTM/GRU handle long-range dependencies.

Chapter 7

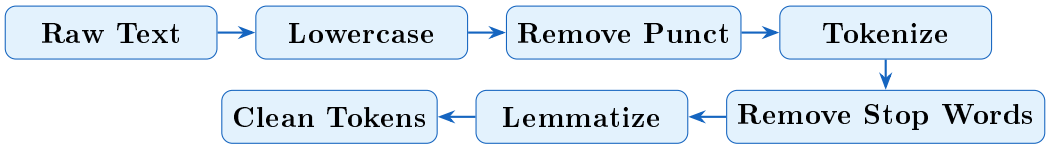
Natural Language Processing (NLP)

7 Natural Language Processing (NLP)

NLP is the branch of AI that deals with **human language** — text and speech. It enables computers to read, understand, and generate language. Examples: Google Translate, Siri, ChatGPT, spam filters.

7.1 Text Preprocessing & Tokenization

Raw text is messy. Before feeding it to any model, we clean it through a **pipeline** of steps:



Step	What it does	Example
Lowercasing	Standardise all text to lowercase	"Hello World" → "hello world"
Tokenization	Split text into individual tokens (words)	"I love NLP" → ["I", "love", "NLP"]
Stop Word Removal	Remove common words with little meaning	Remove "the", "is", "a", "and"
Stemming	Chop off word endings to get root	"running" → "run", "happily" → "happili"
Lemmatization	Reduce to dictionary base form (smarter)	"better" → "good", "ran" → "run"
Punct. Removal	Strip non-alphabetic characters	"Hello!" → "Hello"

Stemming vs Lemmatization: Stemming is faster but crude ("happily" → "happili" — not a real word). Lemmatization is slower but smarter (uses a dictionary). For most tasks, **prefer lemmatization**.

7.2 Word Embeddings — Turning Words into Numbers

If we just count words, the words "king" and "queen" appear to have nothing to do with each other — they are just two different entries in a vocabulary. **Word embeddings** capture the *meaning* of words by placing them in a mathematical space where similar words are close together.

7.2.1 Bag of Words (BoW)

Count how many times each word appears. Simple, but **loses word order and meaning**.

$$\text{"I love NLP"} \rightarrow \begin{bmatrix} 1 & 1 & 1 & 0 & \cdots \end{bmatrix} \quad \text{(vocabulary: [I, love, NLP, hate, ...])}$$

7.2.2 TF-IDF

Not all words are equally important. A word like "the" appears everywhere — it's not helpful. TF-IDF weights each word by how **frequent** it is in the document **but rare** across all documents.

$$\text{TF-IDF}(t, d) = \underbrace{\frac{\text{count of } t \text{ in } d}{\text{total words in } d}}_{\text{TF: how common in this doc?}} \times \underbrace{\log\left(\frac{N}{\text{docs containing } t}\right)}_{\text{IDF: how rare across all docs?}}$$

7.2.3 Word2Vec

A neural network is trained to predict a word from its context (or context from a word). As a side effect, words that appear in similar contexts get **similar vector representations**.

Famous result: $\vec{\text{King}} - \vec{\text{Man}} + \vec{\text{Woman}} \approx \vec{\text{Queen}}$

Method	Key Idea	Vector Size	Context?
BoW	Raw word counts	Vocabulary size	No
TF-IDF	Weighted word counts	Vocabulary size	No
Word2Vec	Neural prediction	100–300	Fixed window
GloVe	Co-occurrence stats	100–300	Global
BERT Embeddings	Deep transformer	768+	Full sentence

In Word2Vec, "bank" always has the same vector. In BERT, "bank" has a *different* vector in "river bank" vs "bank account" — it understands context!

7.3 Sequential Modelling

Language is sequential — the meaning of a word depends on what came before it. "I did not enjoy the movie" vs "I enjoyed the movie." The word "not" changes everything!

Models for sequential data:

- **RNN** — Basic sequential processing; forgets long dependencies.
- **LSTM / GRU** — Better memory; handles longer sequences.
- **Transformer** — Processes all positions in **parallel** using attention; now the dominant approach.

7.4 Language Model Foundations

A language model estimates the **probability of a sequence of words**:

$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{n} P(w_i \mid w_1, \ldots, w_{i-1})$$

In other words: given all the words so far, **what word comes next?** This is exactly what GPT does — it predicts the next token over and over.

Chapter Summary

Chapter 7 summary: NLP processes text through tokenisation, stop word removal, and lemmatisation. BoW and TF-IDF are simple but lose meaning. Word2Vec and BERT capture semantic meaning. Language models predict the next word — the foundation of ChatGPT and all modern LLMs.

Chapter 8

LLM Architecture & Responsible AI

8 LLM Architecture & Responsible AI

8.1 Transformers — The Architecture Behind Modern AI

In 2017, the paper **"Attention Is All You Need"** introduced the **Transformer** architecture. It replaced RNNs and became the foundation of BERT, GPT, ChatGPT, Claude, Gemini, and virtually every modern LLM.

8.1.1 Why Transformers? The Problem with RNNs

- RNNs process words **one at a time** (slow, not parallelisable).
- They struggle to connect words that are **far apart** in a sentence.
- Transformers process all words **simultaneously** and directly relate any word to any other.

8.1.2 Self-Attention Mechanism

This is the core innovation of Transformers. Every word "**looks at**" all other words and decides **how much attention** to pay to each one.

Sentence: "**The animal didn't cross the street because it was too tired.**"
What does *"it"* refer to? "animal" or "street"? A human instantly knows "animal."
Self-attention allows the model to connect "it" strongly to "animal" — the model learns this from data.

For each word, we create three vectors from learned weight matrices:

- **Query (Q):** "What am I looking for?"
- **Key (K):** "What do I have to offer?"
- **Value (V):** "What information should I pass on?"

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

- $\mathbf{Q}\mathbf{K}^T$ — How much does each query match each key? (attention scores)
- $\div\sqrt{d_k}$ — Scale down to prevent vanishing gradients when dimensions are large
- $\text{softmax}(\cdots)$ — Convert scores to probabilities (must sum to 1)
- $\times\mathbf{V}$ — Weighted sum of values = final output

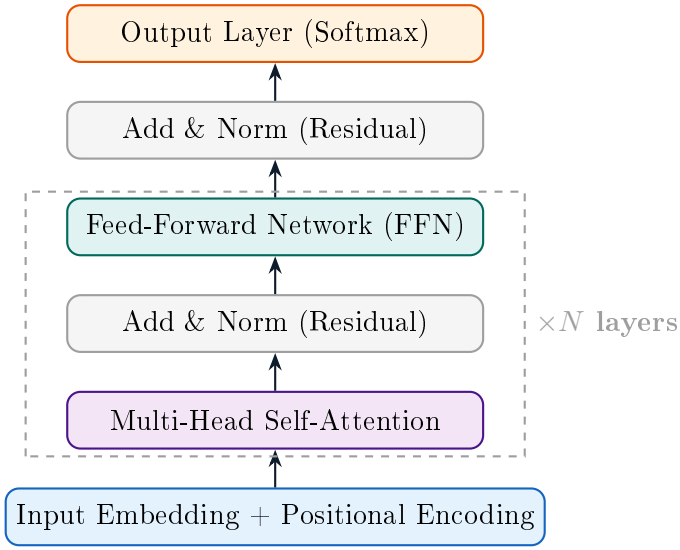
8.1.3 Multi-Head Attention

Instead of computing attention once, the Transformer does it h **times in parallel** ("h heads"), each learning to attend to different aspects of meaning (grammar, semantics, coreference, etc.), then concatenates the results.

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O$$

where $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q,\ \mathbf{K}\mathbf{W}_i^K,\ \mathbf{V}\mathbf{W}_i^V)$

8.1.4 Full Transformer Architecture



Encoder vs Decoder: BERT uses only the **encoder** (great for understanding tasks — classification, NER). GPT uses only the **decoder** (great for text generation). T5 and BART use both.

8.2 Responsible AI & Fairness Principles

As AI systems become more powerful and widely deployed, we must ensure they are **safe, fair, and trustworthy**.

Principle	Meaning in Plain English
Fairness	The model should not discriminate based on race, gender, age, religion, or other protected attributes.
Transparency	We should be able to understand *why* the model made a decision (explainability).
Accountability	There must be a clear person or organisation responsible for the model's decisions.
Privacy	Personal data should be handled respectfully — minimal collection, strong protection (e.g., GDPR).
Safety	The model must not produce harmful, dangerous, or illegal outputs.
Reliability	Performance must be consistent across different demographic groups and contexts.

8.3 Bias in AI — Where Does it Come From?

In 2018, Amazon scrapped an AI hiring tool because it was systematically **downgrading résumés from women**. It had learned this from historical male-dominated hiring data. Bias in ⇒ bias out.

Sources of bias:

- **Data bias:** Training data reflects historical human prejudices.
- **Label bias:** Annotators bring their own biases when labelling data.
- **Model bias:** The model amplifies small biases in data.

8.3.1 Bias Mitigation Strategies

Stage	Type	Techniques
Before training	Pre-processing	Re-sampling, re-weighting, curating fair datasets
During training	In-processing	Fairness constraints in the loss function, adversarial debiasing
After training	Post-processing	Calibrate thresholds per demographic group

8.4 Misinformation Prevention

LLMs can confidently generate text that sounds correct but is **completely false**. For example, fabricating citations, fake news articles, or non-existent laws. This is called **hallucination**.

Technique	How it Helps
RAG (Retrieval-Augmented Generation)	Attach relevant documents from a trusted database before generating — grounds the model in real facts
RLHF (Reinforcement Learning from Human Feedback)	Humans rate outputs; model learns to prefer accurate, helpful, safe responses
Red-Teaming	Adversarially try to make the model say harmful things before deployment; fix the gaps
Watermarking	Embed invisible statistical signals in generated text to detect it later
Content Filters	System-level classifiers that block harmful, false, or dangerous outputs

Chapter Summary

Chapter 8 summary: Transformers use self-attention (QK^TV) to process all words in parallel — far superior to RNNs. Multi-head attention captures different relationships simultaneously. Responsible AI demands fairness, transparency, privacy, and safety. LLMs can hallucinate — RAG and RLHF are key mitigations.

Final Review

Everything in One Place — Quick Reference

Master Summary: All Chapters at a Glance

#	Topic	Core Idea	Key Formula / Fact
1	**ML Foundations**	DL $\subset$ ML $\subset$ AI. Types: Supervised, Unsupervised, Semi, RL	ML: Data+Output $\rightarrow$ Rules
2	**Preprocessing**	Clean data before modelling	$x' = \frac{x-\mu}{\sigma}$ (standardise)
3	**Linear Models**	LR predicts numbers; LogR classifies	$\sigma(z) = \frac{1}{1+e^{-z}}$
4	**Evaluation**	Measure model quality correctly	$F_1 = 2\frac{PR}{P+R}$; $R^2 \in (-\infty, 1]$
5	**Decision Trees**	Split recursively on best feature	Gini $= 1 - \sum p_i^2$
6	**Neural Networks**	Layers of neurons, trained by backprop	$w \leftarrow w - \eta \nabla_w \mathcal{L}$
7	**NLP**	Text $\rightarrow$ clean $\rightarrow$ embed $\rightarrow$ model	King−Man+Woman≈Queen
8	**LLMs & Ethics**	Transformers use self-attention	Attn $= \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Good Luck on Your Exam!

Remember the three keys to success:

Understand **Apply** **Explain Simply**

If you can explain a concept to a ten-year-old, you truly understand it.